1 Lesson 21-10-26

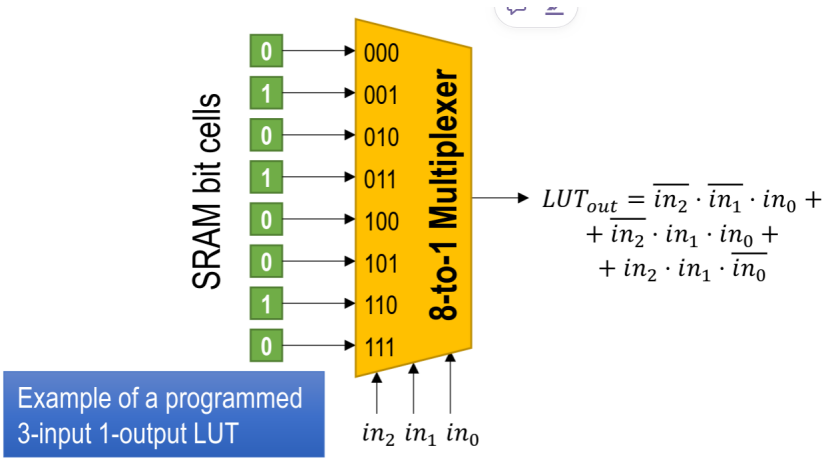
1.0.1 FPGA LUTs

Let's look at FPGA *Look-Up Tables* (**LUT**s) in detail. A LUT basically implements an n-input, 1-output combinational logic circuit, made up of:

- 2^n SRAM bit cells;
- A $2^n \rightarrow 1$ multiplexer, with n selection inputs.

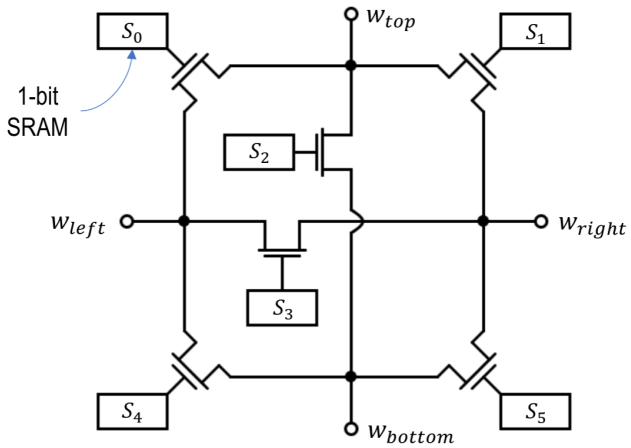
Note that an n-input,-1-output LUT is like a $2^n \times 1$ memory with:

- A single row, so need of a row decoder, and a row that is always selected;
- 2^n columns, where the MUX is the column decoder and column multiplexer of the memory.

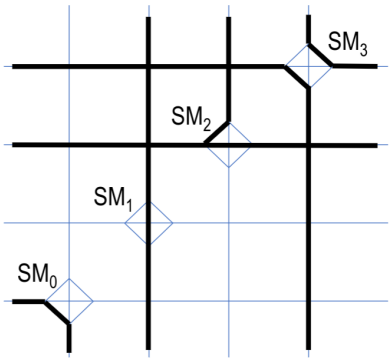


1.0.2 Interconnections

Interconnections in FPGAs are usually implemented via **switch matrices**, usually made up of 6 pass transistors.



It is pretty clear how, by changing the contents of the control SRAM we can change the thresholds of the pass transistors and thus the pairs of nodes across w_{top}, w_{right}, w_{bottom} and w_{left}, creating any kind of interconnection.



	SM_0	SM_1	SM_2	SM_3
S_0	0	0	1	0
S_1	0	0	0/1	1
S_2	0	1	0	0
S_3	0	0	1	0
S_4	1	0	0	1
S_5	0	0	0	0

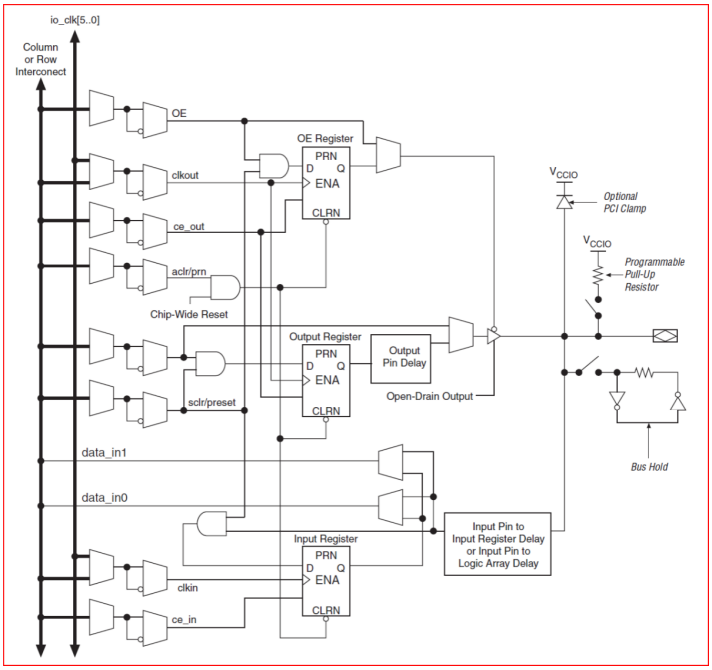
A switch matrix divides wires into independent segments. Segments are connected by properly activating transistors in the switch matriix. If not connected, adjacent segments may belong to different interconnections, with no conflicts because inactive transistors separate them.

Because a switch matrix introduces delays and increases the area, we tend to limit the number of switch matrices to improve performance and decrease costs. On the other hand a switch matrix increases flexibility, and more switch matrices make routing easier.

1.0.3 I/O blocks

FPGAs also feature **I/O blocks** such as the ones saw for CPLDs in 12.1. Because FPGAs are usually more complex, though, the I/O blocks are usually more complex too. In this they have:

- Output buffer with Enable signal;
- Combinational or sequential inputs and outputs;
- Optional Pull-Up, Clamp and Bus Hold circuits, configurable by controlling the selection signals of multiplexers



1.0.4 Additional FPGA blocks

FPGAs, being more complex and versatile than other logic devices, usually feature additional blocks such as:

- **Memories** and **FIFO** (*First-In, First-Out*) queues. These are usually SRAMs, but not control SRAM cells like we saw before: actual SRAM topologies in the order of kilobytes, with address decoders and bit multiplexers. Sometimes interfaces for external dynamic memories are available.
- **Carry chains** between different CLBs or LEs: these allow implementing efficient adders;
- Embedded hardware **multipliers** or **DSP** (*Digital Signal Processing*) blocks;
- Clock management circuitry;
- High speed **I/O**;
- Even **processor cores**.

1.0.5 FPGA performance

Let's do a few performance and design considerations. First off, the kind of technology used to program the FPGA is slighlty varied:

- **SRAM** is dominant, used for LUTs, but also to control MUXes or pass transistors. Controlling standard CMOS technology, it can be reprogrammed on the fly, which makes for a standard technology which allows to easily embed other blocks (which is the reason FPGAs are usually at the cutting-edge of a given technology). The main con is the large size, power consumption, and the fact that SRAM is not permanent;
- **Fuse and Antifuse** is sometimes used as its small and permanent, but it cannot be reprogrammed nor tested;
- **EEPROM** or **Flash** can be reprogrammed, but don't represent a standard proces.

We can then compare to other PLD technologies, namely **CPLD**s.

FPGAs are configured at a finer level than CPLDs, which have few large configurable blocks, while FPGAs have many small configurable blocks. FPGAs are volatile, typically implemented using SRAM technology, and must be reprogrammed at every power-up, whereas CPLDs are non-volatile, typically using EEPROM technology. FPGAs use standard technology, making it easier to include additional resources. They are more expensive, but cheaper per gate, and usually much larger. CPLDs are more predictable, while FPGA delays can vary significantly with the implementation.